

Non-autoregressive Generative Models for Reranking Recommendation

Yuxin Ren
Kuaishou Technology
Beijing, China
renyuxin@kuaishou.com

Qiya Yang
Peking University
Beijing, China
yangqiya@stu.pku.edu.cn

Yichun Wu
Tsinghua University
Beijing, China
wuyc21@mails.tsinghua.edu.cn

Wei Xu
Kuaishou Technology
Beijing, China
xuwei09@kuaishou.com

Yalong Wang
Kuaishou Technology
Beijing, China
wangyalong03@kuaishou.com

Zhiqiang Zhang[†]
Kuaishou Technology
Beijing, China
zhangzhiqiang06@kuaishou.com

ABSTRACT

Contemporary recommendation systems are designed to meet users' needs by delivering tailored lists of items that align with their specific demands or interests. In a multi-stage recommendation system, reranking plays a crucial role by modeling the intra-list correlations among items. The key challenge of reranking lies in the exploration of optimal sequences within the combinatorial space of permutations. Recent research proposes a generator-evaluator learning paradigm, where the generator generates multiple feasible sequences and the evaluator picks out the best sequence based on the estimated listwise score. The generator is of vital importance, and generative models are well-suited for the generator function. Current generative models employ an autoregressive strategy for sequence generation. However, deploying autoregressive models in real-time industrial systems is challenging. Firstly, the generator can only generate the target items one by one and hence suffers from slow inference. Secondly, the discrepancy between training and inference brings an error accumulation. Lastly, the left-to-right generation overlooks information from succeeding items, leading to suboptimal performance.

To address these issues, we propose a Non-AutoRegressive generative model for reranking Recommendation (NAR4Rec) designed to enhance efficiency and effectiveness. To tackle challenges such as sparse training samples and dynamic candidates, we introduce a matching model. Considering the diverse nature of user feedback, we employ a sequence-level unlikelihood training objective to differentiate feasible sequences from unfeasible ones. Additionally, to overcome the lack of dependency modeling in non-autoregressive models regarding target items, we introduce **contrastive decoding** to capture correlations among these items. Extensive offline experiments validate the superior performance of NAR4Rec over state-of-the-art reranking methods. Online A/B tests reveal that

NAR4Rec significantly enhances the user experience. Furthermore, NAR4Rec has been fully deployed in a popular video app Kuaishou with over 300 million daily active users.

CCS CONCEPTS

• **Information systems** → **Retrieval models and ranking**; • **Computing methodologies** → *Machine learning algorithms*.

KEYWORDS

Recommender systems, Generative Model, Non-autoregressive Models

ACM Reference Format:

Yuxin Ren, Qiya Yang, Yichun Wu, Wei Xu, Yalong Wang, and Zhiqiang Zhang[†]. 2024. Non-autoregressive Generative Models for Reranking Recommendation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24)*, August 25–29, 2024, Barcelona, Spain. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3637528.3671645>

1 INTRODUCTION

Recommendation systems offer users personalized item lists tailored to their interests. Various approaches have been proposed to capture user interests, focusing on feature interactions[5, 12, 17], user preference modeling[33, 34], and so on. However, most existing methods treat individual items separately, neglecting their mutual influence and leading to suboptimal results. Acknowledging that user interactions with one item may correlate with others in the recommendation list[23], reranking is introduced to consider contextual information and generate an optimal sequence of recommendation items.

The main challenge in reranking is exploring optimal sequences within the vast space of permutations. Reranking methods are typically categorized into on-stage and two-stage approaches. One-stage methods [1, 23] take candidates as input, estimating refined scores for each item within the permutation, and rerank them greedily based on these scores. However, one-stage methods encounter an inherent contradiction[7, 30]: the reranking operation inherently alters the permutation, introducing different mutual influences between items compared to the initial arrangement. Consequently, the refined score conditioned on the initial permutation is considered implausible.

To tackle this challenge, two-stage methods utilize a generator-evaluator framework. Here, the generator creates multiple feasible

[†] Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

KDD '24, August 25–29, 2024, Barcelona, Spain.

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0490-1/24/08

<https://doi.org/10.1145/3637528.3671645>

sequences, and the evaluator selects the optimal sequence based on the estimated listwise score. Within the generator-evaluator framework, the generator plays a crucial role. Generative models[3, 7, 9, 35] are preferred over heuristic methods[7, 18, 24, 30] for the generator function due to the expansive solution space of item permutations. Generative models commonly employ an autoregressive strategy for sequence generation.

However, deploying the autoregressive models in real-time industrial recommendation systems presents challenges. Firstly, autoregressive models suffer from inference efficiency. Autoregressive models adopt a sequential approach to generate target sequences item by item, resulting in slow inference as the time complexity increases linearly with the sequence length. Secondly, a critical issue arises from the training-inference discrepancy in autoregressive models. While these models are trained to predict the next item based on the ground truth up to that point. However, during inference, they receive their own previously generated outputs as input. This misalignment introduces an accumulated error, where inaccuracies generated in earlier timesteps propagate and accumulate over time. Consequently, this accumulation leads to sequences that deviate from the true distribution of the target sequence. Additionally, autoregressive models have limited information utilization. The sequential decoding process focuses solely on preceding items, neglecting information from succeeding items. This limitation results in suboptimal performance as the model fails to fully leverage the available context.

To address those challenges, we propose **Non-AutoRegressive generative model for reranking Recommendation (NAR4Rec)**. Unlike autoregressive models, which generate sequences step by step, relying on their own previous outputs, NAR4Rec generates all items within the target sequence simultaneously.

However, we find it nontrivial to deploy non-autoregressive in recommendation systems. Firstly, the sparse nature and dynamic candidates in recommendation systems pose difficulties for learning convergence, which we address by sharing position embedding and introducing a matching model. Secondly, the diverse nature of user feedback, including both positive and negative interactions, renders maximum likelihood training less suitable. We propose unlikelihood training to distinguish between desirable and undesirable sequences. Lastly, non-autoregressive models assume an independent selection of items at each position in a sequence, which is inadequate when modeling intra-list correlation. Hence we propose contrastive decoding to capture dependencies across items.

To summarize, our contributions are listed as follows:

- We make the first attempt to adopt non-autoregressive models for reranking, which significantly speeds up the inference speed and meets the requirements of real-time recommendation systems.
- We propose a matching model to enhance convergence, a sequence-level unlikelihood training method to guide the generated sequence towards improved overall utility, and a contrastive decoding method to refine current decoding strategies with intra-list correlation.
- Extensive offline experiments demonstrate that NAR4Rec outperforms state-of-the-art methods. Online A/B tests further validate the effectiveness of NAR4Rec. Furthermore,

NAR4Rec has been fully deployed in a real-world video app Kuaishou with over 300 million daily active users, notably improving the user experience.

2 RELATED WORK

2.1 Reranking in Recommendation Systems

In contrast to earlier phases like matching and ranking[4, 19], which typically learn a user-specific itemwise scoring function, the core of reranking in recommendation systems lies in modeling correlations within the exposed list. Reranking[22, 23, 30], building upon candidate items from the ranking stage, selects a subset and determines their order to ensure exposing the most suitable items to the users. Existing research on reranking can be systematically classified into two principal categories: one-stage[1, 22, 23], and two-stage methods[7, 18, 24, 30].

One-stage methods treat reranking as a retrieval task, recommending the top k items based on scores from a ranking model. These methods refine the initial list distribution using list-wise information, optimizing overall recommendation quality. Subsequently, the candidates are reranked by the refined itemwise score in a greedy manner. The distinction lies in the network architectures for capturing list-wise information, such as GRU in DLCM[1], and transformer in PRM[23]. However, user feedback for the exposed list is influenced not just by item interest but also by arrangements and surrounding context[13, 20, 21, 31]. The reranking operation modifies permutations, thereby introducing influences distinct from the initial permutation. Moreover, one-stage methods, which exclusively model the initial permutation, fall short of capturing alternative permutations. Consequently, those methods struggle to maximize overall user feedback[30].

Two-stage methods [7, 18, 24, 30] embrace a generator-evaluator framework. In this framework, the generator initiates the process by generating multiple feasible sequences, and subsequently, the evaluator selects the optimal sequence based on the estimated listwise score. This framework allows for a comprehensive exploration of various feasible sequences, and an informed selection of the most optimal one based on listwise considerations. The role of the generator is particularly crucial for generating sequences. Common approaches for generators can be categorized into heuristic methods[7, 18, 24, 30], such as beam search or item swapping, and generative models [3, 7, 9, 35]. Generative models are more suitable than heuristic methods for reranking given the vast permutation space. These generative models typically adopt a step-greedy strategy which autoregressively decides the display results of each position. However, the high computational complexity of online inferences limits their application in real-time recommendation systems.

To address the challenges linked with autoregressive generation models, our work investigates the viability of non-autoregressive generative models within the generator-evaluator framework. Non-autoregressive generative models generate the target sequence once to alleviate computational complexity.

2.2 Non-autoregressive Sequence Generation

Non-autoregressive sequence generation[10] was initially introduced in machine translation to speed up the decoding process.

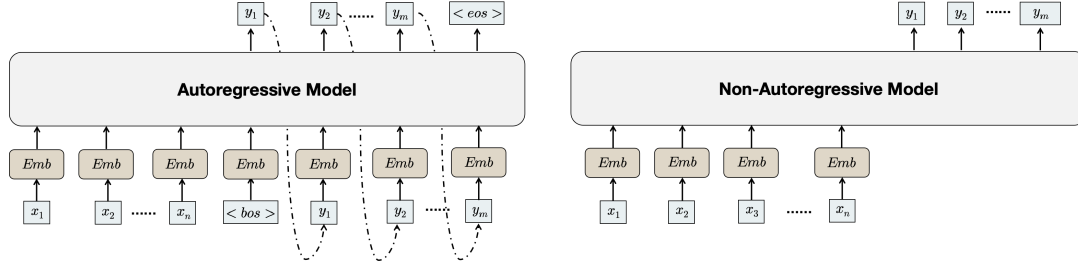


Figure 1: Comparison between autoregressive model (left) and non-autoregressive model (right). Auto-regressive models generate items sequentially while non-autoregressive models generate all items simultaneously.

Then it has since gained increased attention in nature language processing, e.g. text summarization[2, 11], text error correction[14, 15]. Specifically, efforts have been focused on tackling the absence of target information in non-autoregressive models. Strategies include enhancing the training corpus to mitigate target-side dependencies[10, 32] and refining training approaches[8, 25] to alleviate learning difficulties.

Although non-autoregressive generation has been explored in text, those conventional techniques are not directly applicable to recommendation systems. We tackle the challenges encountered in recommendation systems to improve the convergence and performance of non-autoregressive models and make the first attempt to integrate non-autoregressive models into reranking within real-time recommender systems.

3 PRELIMINARY

3.1 Reranking problem Formulation

For each user u within the set U , a request encompasses a set of user profile features (such as user ID, gender, age), the recent interaction history, and n candidates items denoted as $X = \{x_1, x_2, \dots, x_n\}$, where n is the number of candidates. Given candidates X , the goal of reranking is to propose an item sequence $Y = \{y_1, y_2, \dots, y_m\}$ that elicits the most favorable feedback for user u , where m is the sequence length and Y is the recommended list of the reranking model. We denote the reranking models as $\mathcal{F}(u, \theta, X)$ where the corresponding parameter is θ . In real-time recommendation systems, reranking acts as the last stage to deliver the ultimate list of recommended items. Typically, n significantly exceeds m , with m being less than 10 and n ranging from several tens to hundreds.

In a multi-interaction scenario, users may exhibit distinct types of interaction (e.g., clicks, likes, comments) for each item exposure. Formally, we define the set of user interactions as B , and $s_{u, y_i, b}$ represents user u 's response to item y_i concerning interaction $b \in B$. Given Y , each item y_i has a multi-interaction response $\mathbf{e}_{u, y_i} = [e_{u, y_i, b_1}, \dots, e_{u, y_i, b_{|B|}}]$. For all items Y , the overall user response is:

$$\mathbf{E}_{u, Y} = \begin{bmatrix} e_{u, y_1, b_1} & \cdots & e_{u, y_m, b_1} \\ \vdots & \ddots & \vdots \\ e_{u, y_1, b_{|B|}} & \cdots & e_{u, y_m, b_{|B|}} \end{bmatrix}. \quad (1)$$

The overall utility is quantified as the summation of individual item utilities, denoted as $\mathcal{R}(u, Y) = \sum_{i=1}^m \mathcal{R}(u, y_i)$. The utility associated with each item may correspond to a specific interaction type b ,

such as clicks, watch time, or likes. In such cases, the item utility is expressed as $\mathcal{R}(u, y_i) = e_{u, y_i, b}$. Alternatively, the item utility can be represented as the weighted sum of diverse interactions $\mathcal{R}(u, y_i) = \sum_b w_b e_{u, y_i, b}$, where w_b denotes the weight for each interaction.

The reranking objective is to maximum the overall utility $\mathcal{R}(u, Y)$ for a given user u :

$$\max_{\theta} \mathcal{R}(u, Y). \quad (2)$$

Reranking introduces a permutation space with exponential size, represented as $O(A_n^m)$, where n represents the number of candidates and m represents the number of items to be selected and ordered. Each permutation represents a potential arrangement of items, and users provide unique feedback for each permutation. However, in practical scenarios, users typically encounter only one permutation. Thus, the main challenge in reranking lies in efficiently and effectively determining the optimal permutation given the vast solution space yet extremely sparse real user feedback as training samples.

3.2 Autoregressive sequence generation

Given a set of candidate items denoted as X , autoregressive models decompose the distribution over potential generated sequences Y into a series of conditional probabilities:

$$p_{AR}(Y|X; \theta) = \prod_{i=1}^{m+1} p(y_i | y_{0:i-1}, x_{1:n}; \theta), \quad (3)$$

where the special tokens y_0 (e.g., $\langle \text{bos} \rangle$) and y_{m+1} (e.g., $\langle \text{eos} \rangle$) denote the beginning and end of target sequences. Importantly, the length of the generated sequence is predetermined and fixed, unlike variable lengths in text.

Factorizing the sequence generation output distribution autoregressively leads to a maximum likelihood training with a cross-entropy loss at each timestep:

$$\mathcal{L}_{AR} = -\log p_{AR}(Y|X; \theta) = -\sum_{i=1}^{m+1} \log p(y_i | y_{0:i-1}, x_{1:n}; \theta). \quad (4)$$

The training objective aims to optimize individual conditional probabilities. In training, when the target sequence is known, these probabilities are calculated based on earlier target items rather than model-generated ones, enabling efficient parallelization. In inference, autoregressive models generate the target sequence item-by-item sequentially, efficiently capturing the distribution of the

target sequence. This makes them well-suited for the reranking task, particularly considering the vast space of possible permutations.

While autoregressive models have proven effective, deploying them in industrial recommendation systems is challenging. Firstly, their sequential decoding process leads to slow inference, introducing latency that hinders real-time application. Secondly, these models, trained to predict based on ground truth, face a discrepancy during inference when they receive their own generated outputs as input. This misalignment may lead to compounded errors, as inaccuracies generated in earlier timesteps accumulate over time, resulting in inconsistent or divergent sequences that deviate from the true distribution of the target sequence. Thirdly, autoregressive models rely on a left-to-right causal attention mechanism, limiting the expressive power of hidden representations, as each item encodes information solely from preceding items[26]. This constraint impedes optimal representation learning, resulting in suboptimal performance.

3.3 Non-autoregressive sequence generation

To address the aforementioned challenges, non-autoregressive sequence generation eliminates autoregressive dependencies from existing models. Each element's distribution $p(y_i)$ depends solely on the candidates X :

$$p_{\text{NAR}}(Y|X; \theta) = \prod_{i=1}^m p(y_i|x_{1:n}; \theta). \quad (5)$$

Then the loss function for the non-autoregressive model is:

$$\mathcal{L}_{\text{NAR}} = -\log p_{\text{NAR}}(Y|X; \theta) = -\sum_{i=1}^m \log p(y_i|x_{1:n}; \theta). \quad (6)$$

Despite the removal of the autoregressive structure, the models retain an explicit likelihood function. The training of models employs separate cross-entropy losses for each output distribution. Crucially, these distributions can be computed simultaneously during inference, which significantly differs from the sequential process of autoregressive models. This non-autoregressive approach reduces inference latency, thereby enhancing the efficiency of recommendation systems in real-world applications.

4 APPROACH

In this section, we present a detailed introduction of NAR4Rec. We will first discuss our non-autoregressive model structure, which estimates the probability by a matching model in section 4.1. Then, we delve into unlikelihood training, a method aimed at discerning feedback within the recommended sequence in section 4.2. Finally, we propose contrastive decoding to model the dependency in target sequence in section 4.3. The sequence evaluator in our generator-evaluator framework is introduced in section 4.4.

4.1 Matching model

Non-autoregressive models encounter challenges in training convergence due to two main reasons. Firstly, the sparse nature of training sequences presents learning difficulties. Unlike text sequences that often share linguistic structures, recommended lists in training samples seldom have the same exposures, posing challenges for effective learning from limited data. Secondly, during the

reranking stage, identical index for candidates may denote different items, leading to a variable vocabulary as the candidates to be ranked vary across samples.

Conventional models may struggle to handle such variations efficiently. To tackle these challenges, we introduce two key components to our models: a *candidates encoder* for effectively encoding representations of candidates and a *position encoder* to capture position-specific information within the target sequence. Initially, we randomly initialize an embedding for each position in the target sequences. Notably, we share these position embeddings across training data to enhance learning on sparse data. Subsequently, we integrate bidirectional self-attention and cross-attention modules to acquire representations for each position, leveraging information from the candidates.

Additionally, to address the challenge posed by the dynamic vocabulary arising from variations in candidates across samples, we employ a matching mechanism. Specifically, we match each candidate with each position in the target sequence, thereby yielding probabilities for each candidate at every position. In the following, we elaborate on the structure of NAR4Rec.

Given a user u and candidates $X = \{x_1, x_2, \dots, x_n\}$, the hidden representation of x_i is $\mathbf{x}_i \in \mathbb{R}^{d_x}$. We stack $\mathbf{x}_i$ together into matrix $\mathbf{X} \in \mathbb{R}^{n \times d_x}$. Additionally, we randomly initialize an embedding vector for each position j as $t_j \in \mathbb{R}^{d_t}$. Also, we stack t_j into $\mathbf{T} \in \mathbb{R}^{m \times d_t}$. To align the embedding dimension of $\mathbf{X}$ with that of $\mathbf{T}$, we project them into same hidden dimension d by a linear projection layer. Then $\mathbf{X} \in \mathbb{R}^{n \times d}$ and $\mathbf{T} \in \mathbb{R}^{m \times d}$. Consequently, $\mathbf{X}$ is represented as $\mathbf{X} \in \mathbb{R}^{n \times d}$ and $\mathbf{T}$ as $\mathbf{T} \in \mathbb{R}^{m \times d}$.

Candidates Encoder. The candidates encoder adopts the standard Transformer architecture[27] by stacking L Transformer layers. In each layer, the architecture mainly consists of two blocks, a self-attention block and a feed-forward network. An input $\mathbf{X}$ for self-attention block is linearly transformed into query ($\mathbf{Q}_x$), key ($\mathbf{K}_x$) and value ($\mathbf{V}_x$) as follows:

$$\mathbf{Q}_x = \mathbf{X}\mathbf{W}_x^Q, \mathbf{K}_x = \mathbf{X}\mathbf{W}_x^K, \mathbf{V}_x = \mathbf{X}\mathbf{W}_x^V, \quad (7)$$

where $\mathbf{X}^Q$, $\mathbf{X}^K$ and $\mathbf{X}^V$ denote the weight matrices. Then, the self-attention operation is applied as:

$$\text{Attention}(\mathbf{Q}_x, \mathbf{K}_x, \mathbf{V}_x) = \text{Softmax}\left(\frac{\mathbf{Q}_x \mathbf{K}_x^T}{\sqrt{d}}\right) \mathbf{V}_x, \quad (8)$$

For a multi-head version of self-attention mechanism, the input is linearly projected into $\mathbf{Q}_x$, $\mathbf{K}_x$ and $\mathbf{V}_x$ with h times using individual linear projections to small dimensions (e.g. $d_k = \frac{d}{h}$). Finally, the output of self-attention (SAN) is

$$\begin{aligned} \text{SAN} &= [\text{head}_1, \dots, \text{head}_h] \mathbf{W}_x^O, \\ \text{head}_i &= \text{Attention}(\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i). \end{aligned} \quad (9)$$

The feed-forward network is typically placed after the self-attention block,

$$\text{FFN}(\mathbf{X}) = \sigma(\mathbf{X}\mathbf{W}_x^{\text{in}}) \mathbf{W}_x^{\text{out}}, \quad (10)$$

where $\mathbf{W}_x^{\text{out}}$ and $\mathbf{W}_x^{\text{in}}$ denote the weight matrices of the two linear projection.

Position Encoder. The position encoder adopts a similar Transformer architecture as the candidates encoder. The key difference between them is that the position encoder inserts a cross-attention

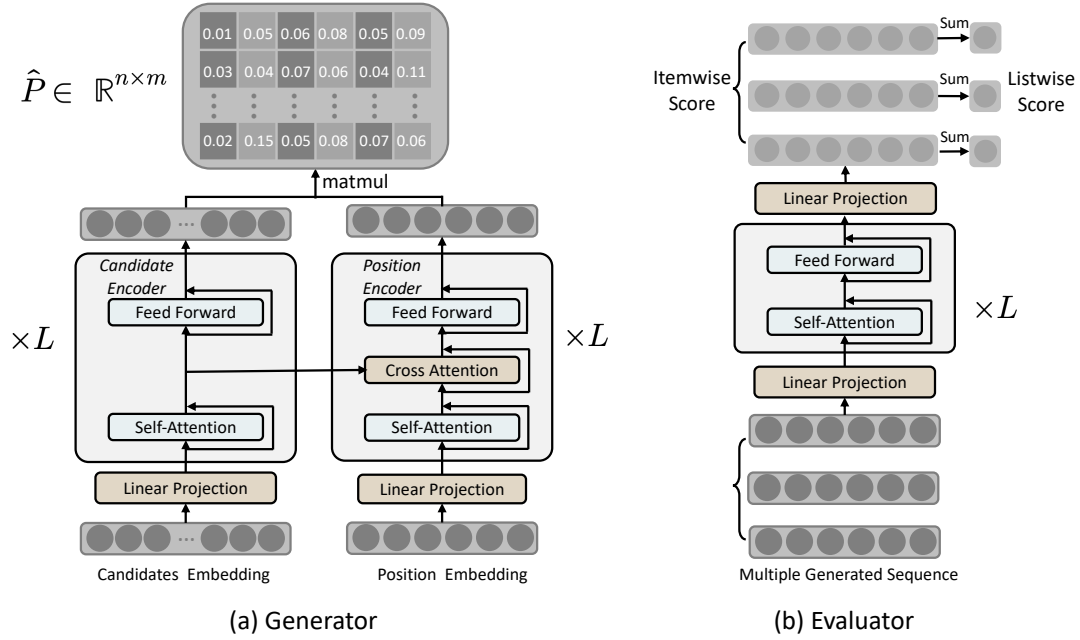


Figure 2: Architectural Overview of the Generator and Evaluator Models. The evaluator evaluates multiple sequences generated by the generator and estimates listwise score to select the optimal sequence.

block between the self-attention block and the feed-forward network in each Transformer layer. As can be seen in Fig. 2, in each layer, the cross-attention block receives the hidden representation from the self-attention blocks of both encoders and processes them via cross-attention operation. Specifically, the hidden representation from the candidates encoder and position encoder are denoted as $\mathbf{X}$ and $\mathbf{T}$, respectively. Similar to the self-attention block, we initially apply linear projection to them:

$$\mathbf{Q} = \mathbf{T}\mathbf{W}^Q, \mathbf{K} = \mathbf{X}\mathbf{W}^K, \mathbf{V} = \mathbf{X}\mathbf{W}^V. \quad (11)$$

Then, we apply the formula in eq. (9) to $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ to get the output hidden representation. The cross-attention is introduced to capture the correlation between candidates and target sequence.

Probability Matrix. To compute the probability matrix, we perform a matrix multiplication on the output hidden representation from the candidates encoder (denoted as $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$) and position encoder (denoted as $\{\mathbf{t}_1, \mathbf{t}_2, \dots, \mathbf{t}_m\}$). Subsequently, we apply a column-wise softmax function to normalize the scores. Formally, the probability score of placing the i -th candidate item to the j -th position is calculated as:

$$\hat{p}_{ij} = \frac{\exp(\mathbf{x}_i^\top \mathbf{t}_j)}{\sum_{i=1}^n \exp(\mathbf{x}_i^\top \mathbf{t}_j)}. \quad (12)$$

Training objectives NAR4Rec is trained via cross-entropy loss function, defined as follows:

$$\mathcal{L}(Y, X) = - \sum_{i=1}^n \sum_{j=1}^m p_{ij} \log(\hat{p}_{ij}), \quad (13)$$

where p_{ij} is 1 if x_j is in position p_j otherwise 0.

4.2 Unlikelihood Training

The discrepancy between text and item sequences hinders the direct application of generative models from text to item recommendation. This disparity arises from the unique characteristics of user interactions in recommendation scenarios. Unlike the structured nature of natural language text, the feedback within recommendation sequences is diverse due to the varied nature of user interactions. While text sequences typically follow conventional language structures to convey information or construct a coherent narrative, user feedback in recommendation sequences is characterized by diverse actions such as clicks or likes, reflecting a diverse and nuanced feedback.

Consequently, the difference in objectives between maximum likelihood training (as in eq. (5)) and reranking (as in eq. (2)) poses a significant challenge. Although maximum likelihood training effectively captures patterns in text sequences, its applicability diminishes in recommendation scenarios where user preferences are dynamic and subjective. The essence of high-quality recommendations lies not just in sequence patterns from training data but, more crucially, in the user utility of the recommended list. User interactions with recommended items are subjective and context-driven, adding complexity to aligning the training objective with the desired model behavior. To address this challenge, we propose unlikelihood training, guiding the model to assign lower probabilities to undesired generations. This adjustment aligns the training process with the intricate feedback patterns.

Unlikelihood training reduces the model's probability of generating a negative sequence. Given candidates X and a negative

sequence Y_{neg} , the unlikelihood loss is:

$$\mathcal{L}_{ul}(Y_{neg}, X) = - \sum_{i=1}^n \sum_{j=1}^m p_{ij} \log(1 - \hat{p}_{ij}), \quad (14)$$

The loss decreases as $\hat{p}_{ij}$ decreases.

Unlike text generation, where messages are clear and content-focused, managing attributes like topic, style, and sentiment in the output text is straightforward[16, 29]. However, recommendation sequences involve user feedback with implicit signals. For instance, a lack of interaction with a recommended item may suggest disinterest. This highlights the model's need to understand both explicit and implicit cues in user feedback. Effective control over generation in recommendation sequences becomes crucial to tailor the output based on user preferences and behaviors, thus enhancing the personalized recommendations. Specifically, we classify a item sequence as positive or negative based on the overall utility defined in section 3.1, and the corresponding loss is as follows:

$$\mathcal{L}_{ul}(Y, X) = \begin{cases} - \sum_{i=1}^n \sum_{j=1}^m p_{ij} \log(1 - \hat{p}_{ij}) & \text{if } \mathcal{R}(u, Y) < \tau \\ - \sum_{i=1}^n \sum_{j=1}^m p_{ij} \log(\hat{p}_{ij}) & \text{if } \mathcal{R}(u, Y) \geq \tau \end{cases} \quad (15)$$

where α is the threshold for positive and negative sequences.

In summary, beyond the primary goal of learning positive sequence patterns through sequence likelihood, unlikelihood training introduces an additional objective to reduce the likelihood of generating sequences with low utilities, effectively training recommendation models to discern feedback within recommendation sequences.

4.3 Contrastive Decoding

Compared with autoregressive generation, the non-autoregressive approach significantly enhances computation efficiency and makes it feasible to deploy in real-time recommendation systems. However, non-autoregressive generation introduce the conditional independence assumption: each target item's distribution $p(y_i)$ depends only on the candidates X . This deviation from autoregressive models poses challenges in capturing the inherently multimodal nature of the distribution of valid target sequences. Take machine translation for example, when translating the phrase "thank you" into German could result in multiple valid translations such as "Vielen Dank" and "Danke". However, non-autoregressive models may generate unpalatable translations like "Danke Dank". The conditional independence assumption in eq. (5) restricts the model's ability to effectively grasp the multimodal distribution in target sequences.

Essentially, the assumption of conditional independence limits the model's ability to navigate a vast solution space and identify the most suitable permutation from numerous valid options for a given set of candidates. This limitation is especially evident in recommendation where the number of reasonable target sequences far exceeds those encountered in text. Consequently, non-autoregressive frameworks grapple with the challenge of mitigating the impact of conditional independence to improve their capacity for generating diverse and contextually appropriate target sequences. To tackle this, we propose contrastive decoding to model the co-occurrence relationship between items and thereby improve the target dependency.

Contrastive decoding incorporates a diversity prior that regulates the sequence decoding procedure. This is grounded in the intuition that an effective recommended list needs to be composed of a wide variety of items. In fact, contrastive decoding leverages a similarity score function as a regulator when decoding, capturing the interdependence between various positions in the target sequence.

Formally, given the preceding context $y_{<i}$, at time step i , the selection of the output y_i follows

$$y_t = \operatorname{argmax}_{x \in X} (1 - \alpha) \times p(x|p_i, X) - \alpha \times \max(s(x, x_j)), \quad (16)$$

where $0 \leq j \leq t - 1$. In eq. (16), the first term, termed as model confidence, denotes the probability of candidates x predicted by the model. The second term, known as similarity penalty, quantifies the distinctiveness candidate x concerning the previously selected items, where $s(x, x_j)$ is computed as:

$$s(x, x_j) = \frac{\mathbf{x}^\top \mathbf{x}_j}{\|\mathbf{x}\| \cdot \|\mathbf{x}_j\|}. \quad (17)$$

Specifically, the similarity penalty is defined as the maximum similarity between the representation of x and all items in $y_{<i}$. NAR4Rec utilizes the dot product item embedding and position embedding to compute the probability matrix. Higher embedding similarity between items often means similar probability in a certain position. We introduce such penalty to introduce intra-list correlation.

Also, to encourage the language model to learn discriminative and isotropic item representations, we incorporate a contrastive objective into the training of the language model. Specifically, given a sequence X , the $\mathcal{L}_{item}$ and $\mathcal{L}_{position}$ are defined as:

$$\mathcal{L}_{item} = \frac{1}{n \times (n - 1)} \sum_{i=1}^n \sum_{j=1, j \neq i}^n \max\{0, \rho - s(\mathbf{x}_i, \mathbf{x}_i) + s(\mathbf{x}_i, \mathbf{x}_j)\}, \quad (18)$$

where $\rho \in [-1, 1]$ is a pre-defined margin and x_i is the hidden representation of item x_i from candidates encoder.

$$\mathcal{L}_{position} = \frac{1}{m \times (m - 1)} \sum_{i=1}^m \sum_{j=1, j \neq i}^m \max\{0, \rho - s(\mathbf{t}_i, \mathbf{t}_i) + s(\mathbf{t}_i, \mathbf{t}_j)\}, \quad (19)$$

where t_i is the hidden representation of position t_i from the position encoder. Intuitively, by training with $\mathcal{L}_{CL}$, the model learns to pull away the distances between representations of distinct tokens.¹ Therefore, a discriminative and isotropic model representation space can be obtained.

The overall training objective $\mathcal{L}$ is then defined as

$$\mathcal{L}(Y, X) = \mathcal{L}_{ul} + \mathcal{L}_{position} + \mathcal{L}_{item}, \quad (20)$$

where the unlikelihood training objective is described in eq. (15). Note that, when the margin ρ in $\mathcal{L}_{position}$ and $\mathcal{L}_{item}$ equals to 0, the $\mathcal{L}(Y, X)$ degenerates to the vanilla unlikelihood training objective $\mathcal{L}_{ul}$.

¹By definition, the cosine similarity $s(h_{x_i}, h_{x_i})$ of the identical token x_i is 1.0.

4.4 Sequence Evaluator

The sequence evaluator model is designed to estimate the overall utility of a given sequence, as illustrated in fig. 2. The generated sequence from the generator is first encoded using a self-attention and a feed-forward layer to capture contextual information. The hidden representation then passes through the linear projection layer to predict the score for a specific target. The overall utility is calculated as the weighted sum of itemwise scores. Ultimately, the sequence with the highest overall utility is chosen for delivery to the users.

5 EXPERIMENTS

In this section, we conduct extensive offline experiments and online A/B tests to demonstrate the effectiveness of NAR4Rec. We first describe our experiment setup and baselines in section 5.1. For offline experiments in section 5.2, we compare NAR4Rec with existing baselines on both performance and training and inference time. Then we alternate the hyper-parameter to analyse the hyper-parameter sensitivity of NAR4Rec. To further show the effectiveness of NAR4Rec in real-time recommendation system, we conduct online A/B tests to ablate our proposed methods in section 5.3.

5.1 Experiment details

Table 1: Statistics of the datasets.

Dataset	#Requests	#Users	#Ads
Avito	53,562,269	1,324,103	23,562,269
Meituan	230,525,531	3,201,922	98,525,531

Dataset: To evaluate reranking recommendation, we expect that each sample of the dataset is an exposed sequence to users rather than a manually constructed sequence. For public dataset, we choose Avito dataset. For industrial dataset, we use real-world data collected from Kuaishou short-video platform. The detailed introduction is given in table 1.

- **Avito**²: The Avito dataset is a publicly available collection of user search logs from avito.ru. The dataset comprises over 53 million lists with 1.3 million users and 36 million ads. Each sample corresponds to a search page with multiple ads. The user search logs from first 21 days are used as training set and the search logs from the last 7 days are used as test set. The sequence length in Avito is 5.
- **Kuaishou**: The Kuaishou dataset is derived from Kuaishou, a widely used short-video application with a user base of over 300 million daily active users. Each sample in the dataset represents an actual user request log, which contains user information (e.g. user id, age, gender), candidates items and user interaction to exposed items. The dataset consists of a total of 82,230,788 users, 26,835,337 items, and 1,811,625,438 requests. Each request contains 6 items in the exposed item sequence and 60 candidates from ranking.

²<https://www.kaggle.com/c/avito-context-ad-clicks/data>

5.2 Offline experiments

Baselines We compare the proposed NAR4Rec with 6 state-of-the-art reranking methods. We select DNN, DCN as pointwise baselines, PRM as one-stage listwise baselines, Edge-Rerank, PIER, Seq2Slate as two-stage baselines. Crucially, Seq2Slate is an autoregressive generative model. A brief overview of these baseline methods is as follows:

- **DNN**[6]: DNN is a basic method for click-through rate prediction, which applies a multi-layer perception to learn feature interaction.
- **DCN**[28]: DCN incorporates feature crossing at each layer, eliminating the need for manual feature engineering while keeping the added complexity to the DNN model minimal.
- **PRM**[23]: PRM models the mutual correlation among items by leveraging the self-attention mechanism and then rank the items by the estimated scores to generate the item sequence.
- **Edge-Rerank**[9]: Edge-Rerank generates the context-aware sequence with adaptive beam search on estimate scores.
- **Seq2Slate**[3]: Seq2Slate leverages pointer networks, which are seq2seq models with an attention mechanism to predict the next item given the items already selected.
- **PIER**[24]: PIER applies hashing algorithm to select top-k candidates from the full permutation based on user interests. Then the generator and evaluator are jointly trained to generate better permutations.

Metrics As there is not common sequence generation metrics for recommendation, we follow previous work[18, 24] and evaluate these models using three commonly adopted metrics: AUC, LogLoss, and NDCG on Avito dataset. For Avito dataset, where n and m are equal(5), the task is to predict the itemwise click-through rate given a listwise input. For Kuaishou dataset, where n and m are 60 and 6 respectively, we employ Recall@6, Recall@10, and LogLoss as evaluation metrics. The task for Kuaishou dataset is to predict whether an item is chosen to be one of the exposed 6 items.

Hyperparameters For Avito dataset, the learning rate is 10-3, the optimizer is Adam and the batch size is 1024. For Kuaishou dataset, the learning rate and optimizer is the same as Avito, but the batch size is 256.

5.2.1 Performance comparison. Here we show the results of our proposed method NAR4Rec. As can be seen in in table 2 and table 3, NAR4Rec outperforms 5 baselines including recent strong reranking methods[3, 23, 24]. PRM outperforms DNN and DCN by effectively capturing the mutual influence between items. Additionally, Edge-rerank surpass DNN and DCN with an adaptive beam search with previous item information. PIER demonstrates superiority over other baselines by the interaction per category. Notably, our proposed method exhibits the highest improvement with a significant increase of 0.0125 in the AUC metric compared to other baseline models. table 3 shows the results of our offline experiments on Kuaishou. The evaluation metrics used in this experiment include Recall@6, Recall@10, and Loss. Our method achieves superior results on all metrics compared to other baseline models as well.

Table 2: Comparison between Nar4Rec and baseline methods on the Avito dataset.

	AUC	LogLoss	NDCG
DNN	0.6614	0.0598	0.6920
DCN	0.6623	0.0598	0.7004
PRM	0.6881	0.0594	0.7380
Edge-rerank	0.6953	0.0574	0.7203
PIER	0.7109	0.0409	0.7401
Seq2Slate	0.7034	0.0486	0.7225
NAR4Rec	0.7234	0.0384	0.7409

Table 3: Comparison between Nar4Rec and baseline methods on KuaiShou.

	Recall@6	Recall@10	LogLoss
DNN	66.47%	86.65%	0.6764
DCN	68.22%	87.95%	0.6809
PRM	73.17%	92.25%	0.5328
Edge-rerank	73.63 %	92.90%	0.5252
PIER	73.50%	92.44%	0.5361
NAR4Rec	74.86%	93.16%	0.5199

5.2.2 Training and Inference Time comparison. Given that NAR4Rec is closely related to autoregressive models, we conduct a comparison with autoregressive models Seq2Slate. We compare the training and inference time on the Avito dataset between Seq2slate and NAR4Rec. We also give training and inference time for generators in other baselines in table 4. Since Seq2slate utilizes recurrent neural networks as its backbone network, both training and inference processes adopt an autoregressive manner. The inference speedup of NAR4Rec over Seq2slate is almost the same as training. NAR4Rec only requires 58 minutes to complete the training while Seq2Slate requires 283 minutes. Such a significant reduction in training time (i.e. approximately 5× speedup) highlights the computational efficiency of NAR4Rec. The autoregressive model represented by Seq2Slate generates target sequences item by item, while our Non-autoregressive NAR4Rec generates all items at once. So when generating a sequence of length 5, NAR4Rec shows approximately 5× speedup.

5.2.3 Hyper-parameter Analysis of NAR4Rec. We further analyze the hyper-parameter sensitivity on NAR4Rec. Here, we conduct a series of experiments on NAR4Rec and PIER. As shown in fig. 3, we demonstrate that our experimental results exhibit insensitivity to variations in the learning rate, batch size, and epoch.

Then, we analyze the impact of weight α and margin ρ in contrastive loss and the impact of penalty parameter α in contrastive decoding. Figure 4 shows the results of our experiments. We change ω while fixing $\rho=0.5$, and change ρ while fixing $\omega=0.01$ in contrastive loss. When changing α in contrastive decoding, we set $\rho=0.5$ and $\omega=0.01$ as the default parameters.

Table 4: The training and inference time comparison between NAR4Rec and baseline methods on the Avito dataset. All experiments are conducted on Tesla T4 16G GPU and the batch size is set to 1024. The training and inference time is calculated by averaging the result over 100 steps.

	Training Time	Inference Time
DNN	0.102s	0.034s
DCN	0.106s	0.035s
PRM	0.109s	0.036s
Edge-rerank	0.105s	0.035s
PIER	0.160s	0.053s
Seq2Slate	0.558s	0.186s
NAR4Rec	0.112s	0.037s

5.3 Online experiments

Text sequence generation often is evaluated by human labeling. In recommendation, we resort to online A/B experiments to obtain the feedback from users to demonstrate our effectiveness.

5.3.1 Experiments setup. In online A/B experiments, we evenly divide the traffic of the entire app into ten buckets. The online baseline is Edge-rerank[9], with 20% of the traffic assigned to NAR4Rec, while the remaining traffic is assigned to Edge-rerank.

5.3.2 Experimental Results. The experiments have been launched on the system for ten days, and the result is listed in table 5. NAR4Rec outperforms Edge-rerank[9] by a large margin. NAR4Rec shows users watch more(i.e a higher Views) videos, spend more time on each video(i.e. more Long Views and Complete Views) and a more positive user feedback(i.e. a improvement on like, follows) over [9].

Views	Likes	Follows	Long Views	Complete Views
+1.161%	+1.71%	+1.15%	+1.82%	+2.45%

Table 5: Online experiments results. All values are the relative improvements of NAR4Rec. For the online A/B test in Kuaishou, the improvements of over 0.5% in positive interactions(like, follow) and 0.2% in views are very significant.

5.3.3 Ablation Study on Unlikelihood Training. To show the effectiveness of unlikelihood training, we compare vanilla training on all exposed sequences with unlikelihood training. Unlikelihood shows more Views and a longer Watch Time.

	Views	Watch Time
Vanilla training	-0.370%*	-0.277%*

Table 6: Online experiments on training methods. All values are the relative changes over Unlikelihood Training. * indicates that the metrics is statistically significant.

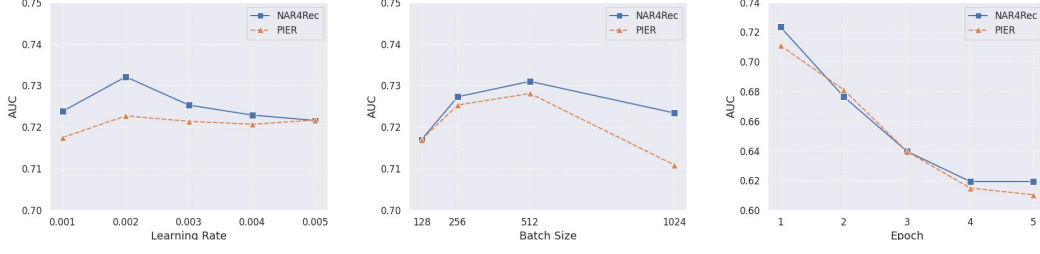


Figure 3: The comparison between NAR4Rec and PIER on Avito with different learning rate, batch size and epoch.

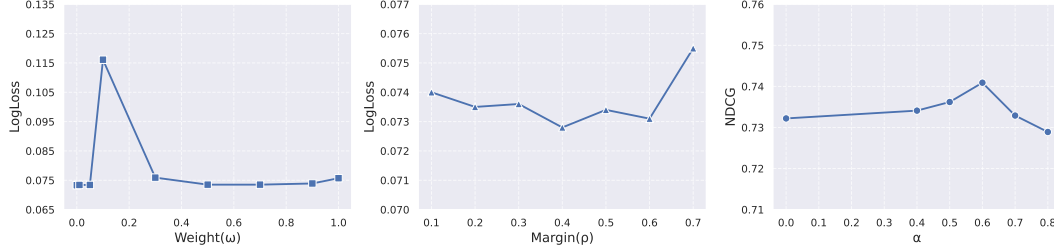


Figure 4: The effect of parameter weight ω and margin ρ in contrastive loss and the α in contrastive decoding.

5.3.4 Ablation Study on Contrastive Decoding. Here we compare the common decoding algorithm in text sequence and a diversity-based algorithm (i.e. Deep DPP) with contrastive decoding. Those decoding algorithms shows a significant drop in View and Watch time, suggesting a poorer user feedback.

	Views	Watch Time
Deep DPP	-0.363%*	-0.361%*
Beam Search	-0.327%*	-0.214%*
Greedy Search	-0.216%*	-0.178%*
Top-k Sampling	-0.254%*	-0.131%

Table 7: Online experiments on decoding methods. All values are the relative changes over the Contrastive Decoding. * indicates that the metrics is statistically significant.

6 CONCLUSION

In this paper, we provide an overview of the current formulation and challenges associated with reranking in recommendation systems. Although non-autoregressive generation has been explored in natural language processing, conventional techniques are not directly applicable to recommendation systems. We tackle the challenges in recommendations to improve the convergence and performance of non-autoregressive models and make the first attempt to integrate non-autoregressive models into reranking in real-time recommender systems. Extensive online and offline A/B experiments have demonstrated the effectiveness and efficiency of NAR4Rec as a versatile framework for generating sequences with enhanced utility. Moving forward, our future work will focus on refining the

modeling of sequence utility to further enhance the capabilities of NAR4Rec.

REFERENCES

- [1] Qingyao Ai, Keping Bi, Jiafeng Guo, and W Bruce Croft. 2018. Learning a deep listwise context model for ranking refinement. In *The 41st international ACM SIGIR conference on research & development in information retrieval*. 135–144.
- [2] Abhijeet Awasthi, Sunita Sarawagi, Rasna Goyal, Sabyasachi Ghosh, and Vihari Piratla. 2019. Parallel Iterative Edit Models for Local Sequence Transduction. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 4260–4270.
- [3] Irwan Bello, Sayali Kulkarni, Sagar Jain, Craig Boutilier, Ed Chi, Elad Eban, Xiyang Luo, Alan Mackey, and Ofer Meshi. 2018. Seq2Slate: Re-ranking and slate optimization with RNNs. *arXiv preprint arXiv:1810.02019* (2018).
- [4] Chris Burges, Tal Shaked, Erin Renshaw, Ari Lazier, Matt Deeds, Nicole Hamilton, and Greg Hullender. 2005. Learning to rank using gradient descent. In *Proceedings of the 22nd international conference on Machine learning*. 89–96.
- [5] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ipsir, et al. 2016. Wide & deep learning for recommender systems. In *Proceedings of the 1st workshop on deep learning for recommender systems*. 7–10.
- [6] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.
- [7] Yufei Feng, Binbin Hu, Yu Gong, Fei Sun, Qingwen Liu, and Wenwu Ou. 2021. GRN: Generative Rerank Network for Context-wise Recommendation. *arXiv preprint arXiv:2104.00860* (2021).
- [8] Marjan Ghazvininejad, Omer Levy, Yinhan Liu, and Luke Zettlemoyer. 2019. Mask-Predict: Parallel Decoding of Conditional Masked Language Models. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 6112–6121.
- [9] Xudong Gong, Qinlin Feng, Yuan Zhang, Jiangling Qin, Weijie Ding, Biao Li, Peng Jiang, and Kun Gai. 2022. Real-time Short Video Recommendation on Mobile Devices. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. 3103–3112.
- [10] Jiatao Gu, James Bradbury, Caiming Xiong, Victor OK Li, and Richard Socher. 2018. Non-Autoregressive Neural Machine Translation. In *International Conference on Learning Representations*.
- [11] Jiatao Gu, Changhan Wang, and Junbo Zhao. 2019. Levenshtein transformer. *Advances in Neural Information Processing Systems* 32 (2019).

- [12] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: a factorization-machine based neural network for CTR prediction. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence*. 1725–1731.
- [13] Thorsten Joachims, Laura Granka, Bing Pan, Helene Hembrooke, and Geri Gay. 2017. Accurately interpreting clickthrough data as implicit feedback. In *Acm Sigir Forum*, Vol. 51. Acm New York, NY, USA, 4–11.
- [14] Yichong Leng, Xu Tan, Rui Wang, Linchen Zhu, Jin Xu, Wenjie Liu, Linquan Liu, Xiang-Yang Li, Tao Qin, Edward Lin, et al. 2021. FastCorrect 2: Fast Error Correction on Multiple Candidates for Automatic Speech Recognition. In *Findings of the Association for Computational Linguistics: EMNLP 2021*. 4328–4337.
- [15] Yichong Leng, Xu Tan, Linchen Zhu, Jin Xu, Renqian Luo, Linquan Liu, Tao Qin, Xiangyang Li, Edward Lin, and Tie-Yan Liu. 2021. Fastcorrect: Fast error correction with edit alignment for automatic speech recognition. *Advances in Neural Information Processing Systems* 34 (2021), 21708–21719.
- [16] Margaret Li, Stephen Roller, Ilia Kulikov, Sean Welleck, Y-Lan Boureau, Kyunghyun Cho, and Jason Weston. 2020. Don't Say That! Making Inconsistent Dialogue Unlikely with Unlikelihood Training. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 4715–4728.
- [17] Jianxun Lian, Xiaohuan Zhou, Fuzheng Zhang, Zhongxia Chen, Xing Xie, and Guangzhong Sun. 2018. xdeepfm: Combining explicit and implicit feature interactions for recommender systems. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1754–1763.
- [18] Xiao Lin, Xiaokai Chen, Chenyang Wang, Hantao Shu, Linfeng Song, Biao Li, et al. 2023. Discrete Conditional Diffusion for Reranking in Recommendation. *arXiv preprint arXiv:2308.06982* (2023).
- [19] Tie-Yan Liu et al. 2009. Learning to rank for information retrieval. *Foundations and Trends® in Information Retrieval* 3, 3 (2009), 225–331.
- [20] Lori Lorigo, Maya Haridasan, Hrönn Brynjarsdóttir, Ling Xia, Thorsten Joachims, Geri Gay, Laura Granka, Fabio Pellacini, and Bing Pan. 2008. Eye tracking and online search: Lessons learned and challenges ahead. *Journal of the American Society for Information Science and Technology* 59, 7 (2008), 1041–1052.
- [21] Lori Lorigo, Bing Pan, Helene Hembrooke, Thorsten Joachims, Laura Granka, and Geri Gay. 2006. The influence of task and gender on search and evaluation behavior using Google. *Information processing & management* 42, 4 (2006), 1123–1131.
- [22] Liang Pang, Jun Xu, Qingyao Ai, Yanyan Lan, Xueqi Cheng, and Jirong Wen. 2020. Setrank: Learning a permutation-invariant ranking model for information retrieval. In *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval*. 499–508.
- [23] Changhua Pei, Yi Zhang, Yongfeng Zhang, Fei Sun, Xiao Lin, Hanxiao Sun, Jian Wu, Peng Jiang, Junfeng Ge, Wenwu Ou, et al. 2019. Personalized re-ranking for recommendation. In *Proceedings of the 13th ACM conference on recommender systems*. 3–11.
- [24] Xiaowen Shi, Fan Yang, Ze Wang, Xiaoxu Wu, Muzhi Guan, Guogang Liao, Wang Yongkang, Xingxing Wang, and Dong Wang. 2023. PIER: Permutation-Level Interest-Based End-to-End Re-ranking Framework in E-commerce. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4823–4831.
- [25] Mitchell Stern, William Chan, Jamie Kiros, and Jakob Uszkoreit. 2019. Insertion transformer: Flexible sequence generation via insertion operations. In *International Conference on Machine Learning*. PMLR, 5976–5985.
- [26] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1441–1450.
- [27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [28] Ruoxi Wang, Bin Fu, Gang Fu, and Mingliang Wang. 2017. Deep & cross network for ad click predictions. In *Proceedings of the ADKDD'17*. 1–7.
- [29] Sean Welleck, Ilia Kulikov, Stephen Roller, Emily Dinan, Kyunghyun Cho, and Jason Weston. 2019. Neural Text Generation With Unlikelihood Training. In *International Conference on Learning Representations*.
- [30] Yunjia Xi, Weiwen Liu, Xinyi Dai, Ruiming Tang, Weinan Zhang, Qing Liu, Xiuqiang He, and Yong Yu. 2021. Context-aware reranking with utility maximization for recommendation. *arXiv preprint arXiv:2110.09059* (2021).
- [31] Ziyang Yang. 2017. Relevance judgments: Preferences, scores and ties. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1373–1373.
- [32] Chunting Zhou, Jiatao Gu, and Graham Neubig. 2019. Understanding Knowledge Distillation in Non-autoregressive Machine Translation. In *International Conference on Learning Representations*.
- [33] Guorui Zhou, Na Mou, Ying Fan, Qi Pi, Weijie Bian, Chang Zhou, Xiaoqiang Zhu, and Kun Gai. 2019. Deep interest evolution network for click-through rate prediction. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33. 5941–5948.
- [34] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep interest network for click-through rate prediction. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1059–1068.
- [35] Tao Zhuang, Wenwu Ou, and Zhirong Wang. 2018. Globally optimized mutual influence aware ranking in e-commerce search. *arXiv preprint arXiv:1805.08524* (2018).